Ain Shams University, Cairo, Egypt
Faculty of Engineering,
International Credit Hours Engineering Programs (ICHEP)

Desktop Hotel Reservation System

Milestone 2 Report

Submitted to:-

Dr. Mahmoud Khalil
Eng. Mario Remon
Eng. Mazen Tarek

Prepared by:-

Yassin Mohamed Yehia (25p0006)
Seif Hazem (25p0005)
Abdullah Alaa (25p0139)
Mohamed Alaaeldin (25p0202)
Karim Ismail Samy (25P0060)

Date of submission: -

7/5/2026

GitHub Repository:

https://github.com/YassinMohamed07/ProjectOOPHotelSystem

YouTube Demo Video:

https://youtu.be/xBq9s3OUMR8

1 Introduction & Objective

This report documents the implementation of Milestone 2 for the Desktop Hotel Reservation System. Building upon the solid Object-Oriented backend developed in Milestone 1, the objective of this phase was to deliver a fully functional, interactive Graphical User Interface (GUI) using the JavaFX framework.

To ensure a clean MVC architecture, we utilized **over 3 FXML layout files** (including `LoginRegister.fxml`, `GuestDashboard.fxml`, `AdminDashboard.fxml`, etc.) to strictly separate the UI design from the backend Java controllers. Furthermore, our team successfully implemented all advanced bonus features, including multi-threading for asynchronous background operations, basic client-server networking (Sockets) for realtime chat, and integration with a persistent real database.

2 Team Responsibilities & Contributions

The workload for Milestone 2 was divided to ensure fair contribution, with complex architectural UI tasks and bonus features assigned strategically to members with the requisite technical bandwidth.

- **Yassin Mohamed Yehia (25p0006) – Lead UI Architect:** Handled the largest and most complex chunk of the GUI. Designed the MVC architecture, implemented complex controllers (`ReceptionistDashboardController`, `AdminDashboardController`, `RoomBrowsingController`), and managed the overarching CSS styling and dynamic JavaFX layout integrations.

- **Seif Hazem (25p0005) – Bonus & Advanced Features Lead:** Designed and implemented all three bonus requirements: 1) Multi-threading logic (using JavaFX Task and `Platform.runLater`); 2) The complete Client-Server networking architecture (`ChatServer`, `ChatClient`) for real-time live chat; 3) Connecting the application to a Real Database instead of relying on in-memory storage.

- **Abdullah Alaa (25p0139) – Moderate GUI Logic:** Developed the `CheckoutController` and `ReservationController`. Implemented complex TableView data bindings, dynamic invoice generation, and the suite of JavaFX Alert dialogues for error/success handling.

- **Mohamed Alaaeldin AlSharkawy (25p0202) – UI Integration:** Developed the `GuestDashboardController`. Ensured seamless data binding between the active Guest model and the profile/balance UI components.

- **Karim Ismail Samy (25P0060) – Foundational UI & Nav:** Implemented the `LoginRegisterController`. Built the `SceneNavigator` utility for seamless view switching using a shared StackPane, and assisted with integration testing.

3 Advanced Features (Bonus Implementations)

3.1 Multi-threading for Background Operations

To ensure a smooth, non-freezing user experience, we integrated Java multithreading using JavaFX's Task class.

- **Background Data Loading:** In the `RoomBrowsingController` and `ReservationController`, room searches and reservation history fetching are executed on background daemon threads.
- **Thread Safety:** We strictly adhered to JavaFX thread-safety rules. Background threads never update the UI directly; instead, `Task.setOnSucceeded()` and `Platform.runLater()` are utilized to push data to the TableViews once the background computation is complete.

3.2 Basic Networking (Client-Server Chat)

We implemented a live chat feature allowing logged-in guests to communicate with the receptionist in real time.

- **Architecture:** A `ChatServer` class acts as the centralized hub using `ServerSocket`. It handles multiple concurrent clients by spawning a new `ClientHandler` thread for every connection.
- **Integration:** The `GuestDashboardController` and `ReceptionistDashboardController` instantiate a `ChatClient` (using Java Sockets) upon initialization. Messages are broadcasted securely across the network and appended to the JavaFX TextAreas dynamically.

3.3 Real Database Integration

Moving beyond the initial Milestone 1 in-memory data store, we connected the application to a real, persistent database. This ensures that all hotel data; including user credentials, room status, reservations, and invoices—are securely saved and retrieved across application restarts, ensuring data persistence and real-world applicability.

4 GUI Design & Screens Implementation

The application strictly utilizes JavaFX (no Swing) with FXML for layout definition, CSS for professional styling (dark theme with vibrant accents), and Java Controllers for logic.

4.1 Login & Registration

Handles both Guest and Staff authentication using `InvalidCredentialException` handling. Includes robust registration validation for password strength and age verification.

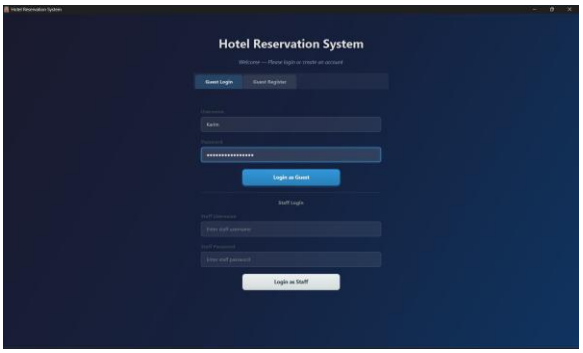

Figure 1: User Login Interface

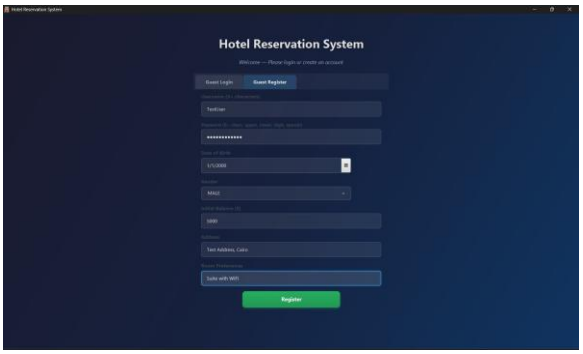

Figure 2: Guest Registration Form

4.2 Guest Operations

The Guest interface allows users to view their dashboard, browse rooms with dynamic filters (Type, Max Price, Dates), add extra paid amenities, view history, and checkout.

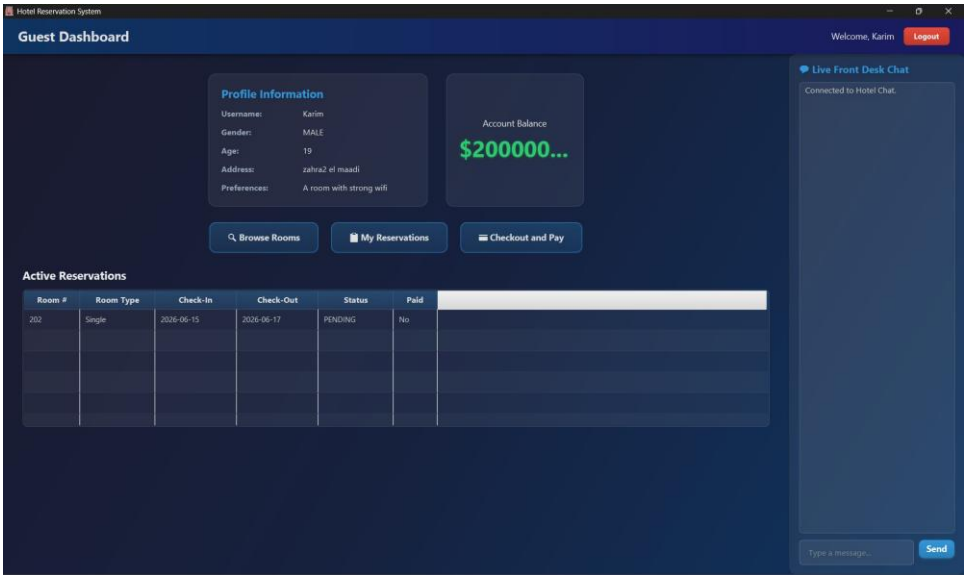

Figure 3: Guest Dashboard displaying profile info, active bookings, and balance.

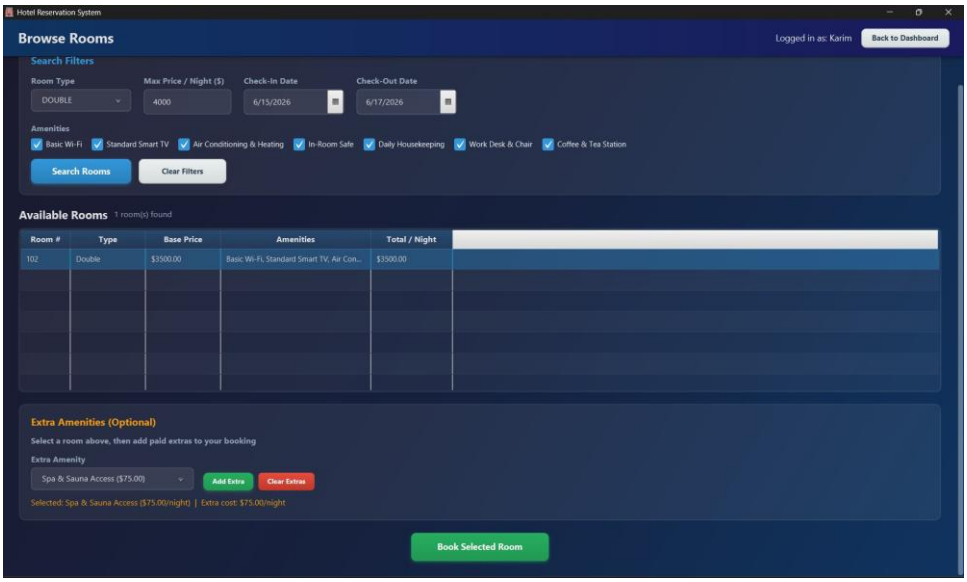

Figure 4: Room Browsing Screen with dynamic filters and optional extra amenities selection.

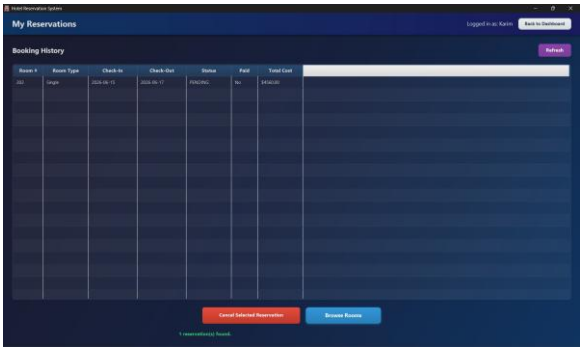

Figure 5: Reservation Management (Booking History)

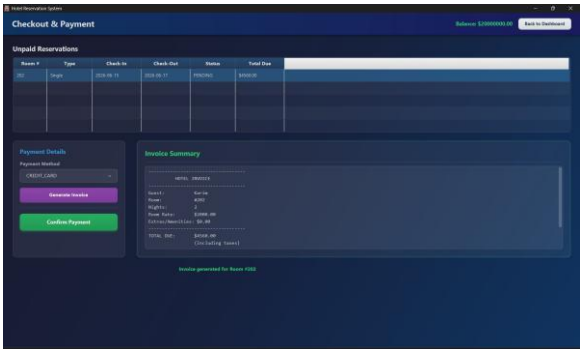

Figure 6: Checkout & Payment with Auto-Invoice

4.3 Staff Operations (Admin & Receptionist)

Staff dashboards utilize complex `TableView` components to manage the hotel's backend entities. The **Admin** manages Rooms, Amenities, and registers new staff. The **Receptionist** handles Check-ins, Check-outs, and payment processing.

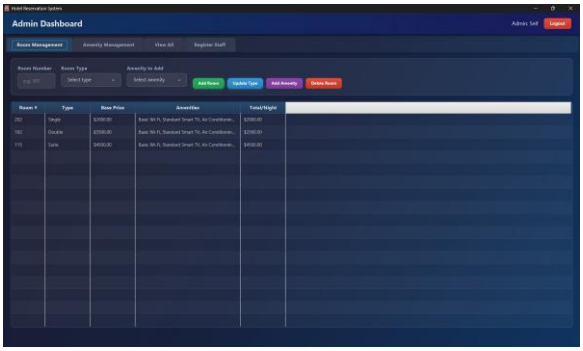

Figure 7: Admin: Room Management

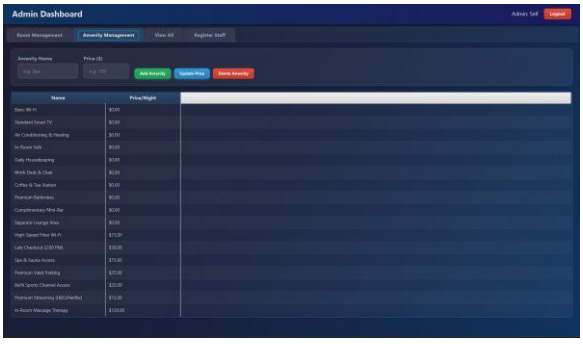

Figure 8: Admin: Amenity Management

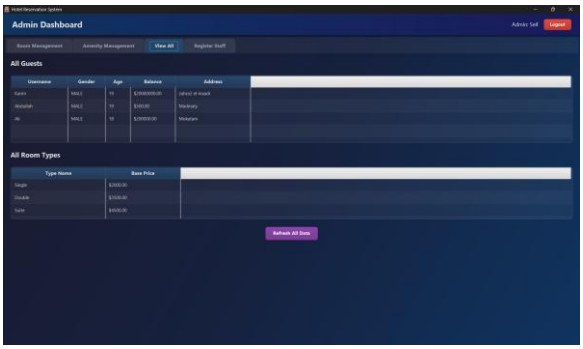

Figure 9: Admin: View All System Data

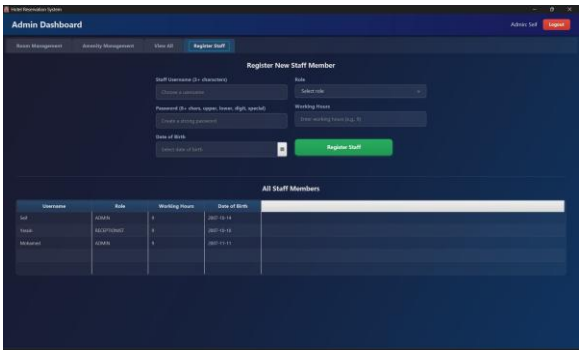

Figure 10: Admin: Staff Registration

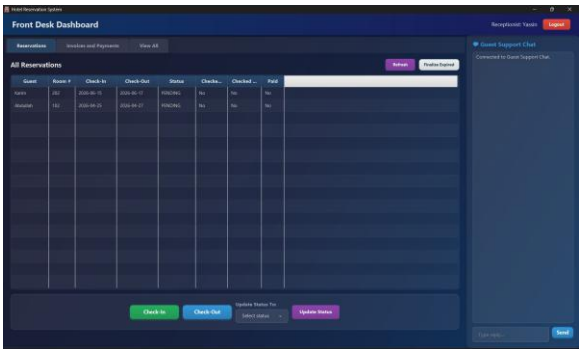

Figure 11: Receptionist: Managing Reservations

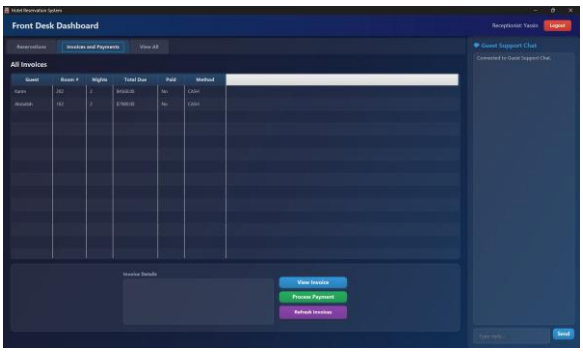

Figure 12: Receptionist: Invoice & Payments

4.4 Live Chat & Error Handling Demonstrations

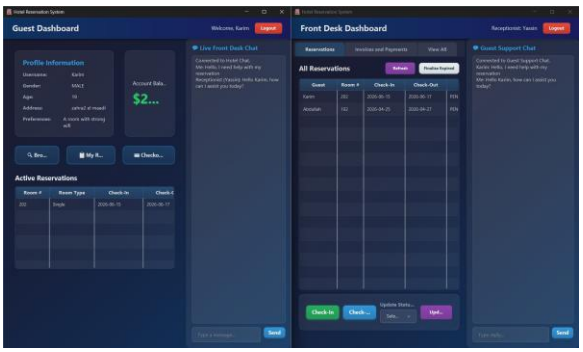

Figure 13: Live Socket Chat between Guest and Receptionist

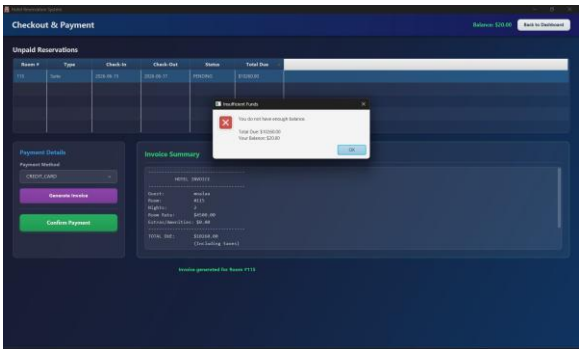

Figure 14: JavaFX Alert: Insufficient Funds Validation

5 Test Cases (Valid & Invalid Scenarios)

Extensive testing was conducted linking the UI elements to the custom Backend Exceptions (`InvalidCredentialException`, `WeakPwordException`, `InvalidDateException`).

Feature	Scenario (Input)	Expected Output / Behavior	Status
Registration	Valid data, strong password (`Pass@123`), Age 19.	Success alert; user added to database.	Pass
Registration	Weak password (`password123` - no special char).	Red error label: *"Password must contain at least one special character"*.	Pass
Registration	Age under 18 (e.g., DOB is 2015).	Red error label: *"Must be 18+ to register"*.	Pass
Login	Incorrect username or password.	Red error label: *"Wrong password"* or *"User not found"*.	Pass
Booking	Check-in date is after Check-out date.	Warning Alert: *"Check-in must be before check-out"*.	Pass
Booking	Dates conflict with existing reservation.	Room does not appear in TableView results.	Pass
Checkout	User tries to pay invoice but balance is lower than total due.	Error Alert: *"You do not have enough balance."*	Pass
Admin	Adds an amenity that already exists.	Status Label (Red): *"Amenity already exists!"*	Pass
Receptionist	Tries to check-in a user before their actual check-in date.	Status Label (Red): *"Cannot check in. Reservation starts on [Date]"*	Pass